snapshots/2-2/sidekiq-book/app/jobs/complete_order_job.rb

```
    def perform(order_id)
      order = Order.find(order_id)
      OrderCreator.new.complete_order(order)
»   rescue BaseServiceWrapper::HTTPError => ex
»     raise IgnorableExceptionSinceSidekiqWillRetry.new(ex)
    end
  end
```

You'll see the code for `IgnorableExceptionSinceSidekiqWillRetry` in a moment. Next, configure the error catcher to ignore this new exception. How you do this highly depends on the error catcher you have set up. Usually, you add the exception's class to some list in a configuration block. For the mock error catcher, add this line of code in `config/initializers/error_catcher.rb`:

snapshots/2-2/sidekiq-book/config/initializers/error_catcher.rb

```
  Rails.application.config.to_prepare do
»   ErrorCatcherServiceWrapper.ignored_errors << IgnorableExceptionSinceSidekiqWillRetry
  end
```

The last step is to implement the new exception. Since it's only relevant to Sidekiq jobs, put it in `app/jobs`:

snapshots/2-2/sidekiq-book/app/jobs/ignorable_exception_since_sidekiq_will_retry.rb

```
  class IgnorableExceptionSinceSidekiqWillRetry < StandardError
    def initialize(exception)
      super(exception.message)
      @cause = exception
    end

    def cause = @cause
  end
```

You'll need to restart your server to try this out; however, it will make this behavior easier to see if we reset the error catcher and clear out Redis as well. When you stop the server, run `bin/rails dev:reset`. This Rake task will reset the fake API and Redis clean states. Then, start the server with `bin/dev` as usual.